

VOYTEK

DOCUMENTAÇÃO TÉCNICA DE SOFTWARE

MVP da plataforma Voytek - infraestrutura financeira e operacional para agentes de IA

Rodrigo Tabaldi
Versão 1.0

Rondonópolis - MT
2026

GUIA DE USO E SUMÁRIO

Documento técnico estruturado no modelo STS fornecido, com apresentação A4, organização por requisitos, arquitetura, dados, segurança, testes, implantação e decisões arquiteturais. O documento descreve a base técnica do MVP da Voytek e deve evoluir junto com o código.

SUMÁRIO

Seção	Conteúdo
1 INTRODUÇÃO E ESCOPO	Problema, objetivo, público, limites e responsáveis.
2 REQUISITOS E RASTREABILIDADE	Requisitos funcionais e não funcionais verificáveis.
3 ARQUITETURA E CENÁRIOS	C4, componentes, fluxos essenciais e estratégia.
4 DADOS, CONTRATOS E INTEGRAÇÕES	Entidades, API, eventos e integrações.
5 QUALIDADE, SEGURANÇA E TESTES	Controles, riscos e plano de testes.
6 IMPLANTAÇÃO E OPERAÇÃO	Azure, deploy, observabilidade e suporte.
7 DECISÕES E REFERÊNCIAS	ADRs e checklist técnico.

1 INTRODUÇÃO E ESCOPO

1.1 Identificação e objetivo

Item	Definição
Sistema ou feature	Voytek MVP - plataforma SaaS para governança econômica e operacional de agentes de IA.
Problema atual	Agentes de IA podem propor ou executar ações empresariais, mas empresas precisam controlar orçamento, finalidade, permissões, autonomia, aprovações e rastreabilidade antes de permitir ações sensíveis.
Objetivo	Permitir que uma empresa crie agentes, associe objetivos e budgets, aplique policies determinísticas e obtenha decisões ALLOW, DENY ou HUMAN_APPROVAL, com Shadow Mode, ledger, outcomes e auditoria.
Público	Empresas que utilizam agentes de IA; administradores; gestores; operadores; equipes financeiras, de TI e automação; sistemas internos e agentes externos integrados por API.
Responsáveis	Produto e arquitetura inicial: Rodrigo Tabaldi. Operação, segurança e revisão técnica serão formalizadas conforme o projeto ganhar equipe e pilotos.

1.2 Escopo, exclusões e premissas

Dentro do escopo	Fora do escopo	Premissas / dependências
Identity e multi-tenancy; Agents; Objectives; Budgets; Policy Engine; autonomia; approvals; Shadow Mode; Ledger; Outcome Ledger; Agent-to-Agent Commerce lógico; AI Orchestrator; Audit; dashboard e API.	Custódia de dinheiro; PIX próprio; cartão próprio; FX real; blockchain; cripto; banco próprio; Kubernetes; antifraude por ML próprio; marketplace público.	C#/.NET; React/TypeScript; PostgreSQL; Azure; LLM via provedor externo; budgets inicialmente lógicos; integrações financeiras reais apenas em fases futuras.

1.3 Controle de versão

Versão	Data	Autor	Alteração	Aprovação
1.0	2026-09-19	Rodrigo Tabaldi	Especificação técnica inicial do MVP.	Em evolução

2 REQUISITOS E RASTREABILIDADE

2.1 Requisitos funcionais

ID	Requisito	Prioridade	Critério de aceite
RF-01	Cadastrar e autenticar usuários e empresas.	Alta	Usuário cria conta, autentica e acessa somente tenants autorizados.
RF-02	Criar, editar, ativar, suspender e consultar agentes.	Alta	Agente persiste com TenantId, status e nível de autonomia.
RF-03	Associar objetivos mensuráveis a agentes.	Alta	Objetivo possui agente, status, período e descrição.
RF-04	Criar budgets lógicos por agente/objetivo.	Alta	Sistema calcula total, reservado, gasto e disponível sem usar ponto flutuante.
RF-05	Avaliar uma AuthorizationRequest no Policy Engine.	Crítica	Toda solicitação retorna ALLOW, DENY ou HUMAN_APPROVAL e registra o motivo.
RF-06	Configurar níveis MANUAL, SUPERVISED e AUTONOMOUS.	Alta	Nível altera o fluxo sem permitir bypass das regras globais.
RF-07	Criar e decidir ApprovalRequest.	Alta	Usuário autorizado aprova/rejeita uma única vez, com auditoria.
RF-08	Executar ações em Shadow Mode.	Crítica	Pipeline completo funciona sem chamar executor externo real.
RF-09	Registrar Ledger append-only.	Crítica	Eventos financeiros não são alterados; correções usam lançamentos compensatórios.
RF-10	Registrar Outcome Ledger.	Média	Ação/objetivo pode receber métrica, valor, unidade e descrição do resultado.
RF-11	Criar CommerceRequest entre agentes.	Média	Solicitação lógica passa pelo Policy Engine e possui ciclo de estados.
RF-12	Integrar IA por ILLMProvider.	Alta	LLM propõe ação estruturada; Policy Engine mantém autoridade final.
RF-13	Registrar AuditLog.	Alta	Mudanças administrativas e decisões sensíveis possuem ator, recurso, horário e correlationId.
RF-14	Expor dashboard e consultas operacionais.	Alta	Usuário visualiza agentes, budgets, decisões, approvals, ledger e Shadow Mode do próprio tenant.

2.2 Requisitos não funcionais

ID	Atributo	Meta objetiva	Evidência
RNF-01	Isolamento	Nenhum tenant acessa dados de outro tenant.	Testes de integração cross-tenant.
RNF-02	Idempotência	Reenvio de operação sensível não duplica efeito.	Teste com mesma Idempotency-Key.
RNF-03	Segurança	Autorização validada no backend; secrets fora do repositório.	Testes, revisão e configuração.
RNF-04	Escalabilidade	API stateless e preparada para múltiplas réplicas.	Teste de execução com réplicas independentes.
RNF-05	Observabilidade	Requests e fluxos críticos possuem correlationId e logs estruturados.	Logs e traces.

ID	Atributo	Meta objetiva	Evidência
RNF-06	Resiliência	Integrações externas usam timeout e política de retry segura.	Testes de falha simulada.
RNF-07	Dados	Valores monetários usam decimal/numeric e transações.	Testes de precisão e concorrência.

2.3 Matriz de rastreabilidade

Requisito	Design / ADR	Teste	Evidência
RF-05	ADR-004 Policy Engine determinístico	PolicyDecisionTests	ALLOW/DENY/HUMAN_APPR OVAL esperado.
RF-08	ADR-005 Shadow Mode	ShadowModeSafetyTests	Nenhum executor real é chamado.
RF-09	ADR-008 Ledger append-only	LedgerAppendOnlyTests	Entradas antigas permanecem imutáveis.
RNF-01	ADR-003 Multi-tenant compartilhado	TenantIsolationTests	Tenant A não lê Tenant B.
RNF-02	ADR-010 Idempotência	IdempotencyTests	Mesmo comando retorna efeito único.

3 ARQUITETURA E CENÁRIOS

3.1 Contexto C4

Item	Definição
Atores	Owner, Admin, Manager, Operator, Viewer; agentes Voytek; agentes/sistemas externos integrados por API.
Fronteira	Pertencem à Voytek: web app, API, módulos de domínio, workers, Policy Engine, ledger, audit e dados. Provedores de LLM, e-mail, storage e futuros rails financeiros são externos.
Integrações	LLM por API; e-mail/notificação; Azure; PostgreSQL; futuros provedores de pagamentos, cartões, Open Finance e identidade.
Diagrama textual	Usuário/Agente -> React/API -> Application -> Policy Engine/Modules -> PostgreSQL; Workers e LLM adapters operam por contratos controlados.

3.2 Contêineres e componentes

Elemento	Responsabilidade	Tecnologia	Entradas / saídas
Web	Interface administrativa e operacional.	React + TypeScript + Vite	HTTPS/JSON.
API	Autenticação, endpoints, orchestration e autorização.	ASP.NET Core / .NET	REST, events internos.
Módulos	Identity, Agents, Objectives, Budgets, Policies, Approvals, Ledger, Outcomes, Commerce, AI, Audit.	C# / DDD pragmático	Commands, queries, domain events.
Workers	Jobs e processamento assíncrono.	C#/ .NET	Eventos, filas futuras, APIs externas.
Banco	Persistência transacional.	PostgreSQL + EF Core	Leitura, escrita, transações, backup.
Storage	Arquivos e artefatos.	Azure Blob Storage	Objetos/arquivos.
LLM	Interpretação e propostas estruturadas.	GPT-5.6 Luna Low como opção padrão otimizada; provider abstrato	Prompt/contexto -> proposta estruturada.

3.3 Cenários essenciais

Cenário	Passos	Falha esperada e resposta
Autorização	Agente propõe ação -> validação -> contexto -> Policy Engine -> ALLOW/DENY/HUMAN_APPROVAL -> ledger/audit.	Budget insuficiente ou policy bloqueia: DENY; inconsistência: sem execução e log correlacionado.
Aprovação humana	Decision=HUMAN_APPROVAL -> cria ApprovalRequest -> usuário autorizado decide -> fluxo continua ou encerra.	Aprovação repetida: idempotência/concorrência impede segunda decisão.
Shadow Mode	Mesmo pipeline -> executor simulado -> registra decisão e impacto estimado.	Qualquer tentativa de executor real é bloqueada pelo modo de execução.
Falha LLM	Orchestrator chama provider com timeout.	Timeout/circuit breaker; nenhuma autorização financeira é concedida por fallback de IA.
Concorrência de budget	Reserva transacional antes da execução.	Conflito causa retry seguro ou rejeição; nunca overspending.

3.4 Estratégia e alternativas

Item	Definição
Abordagem	Modular Monolith com Clean Architecture e módulos de domínio; API stateless; workers separados quando necessário.
Alternativas	Microserviços físicos desde o início: rejeitados por custo e complexidade operacional. Kubernetes: rejeitado no MVP. Banco por serviço: rejeitado inicialmente.
Restrição	Equipe pequena, necessidade de baixo custo, produto ainda em validação, requisitos de segurança e auditabilidade elevados.

4 DADOS, CONTRATOS E INTEGRAÇÕES

4.1 Dados relevantes

Dado / entidade	Classificação	Origem e retenção	Regra
Tenant/User/Membership	Interno / pessoal	Cadastro e autenticação; conforme política futura de retenção.	Acesso por membership e RBAC.
Agent/Objective	Interno	Criado pelo tenant.	Sempre associado a TenantId.
Budget/Reservation	Financeiro interno	Gerado pela aplicação.	decimal/numeric; transacional; auditável.
Policy/Decision	Interno crítico	Configuração e execução do Policy Engine.	Versionar alterações relevantes; registrar matched policies.
Ledger	Financeiro interno crítico	Eventos da plataforma.	Append-only.
AuditLog	Segurança	Ações administrativas.	Append-only; acesso restrito.
LLM context/output	Interno; pode conter dado sensível	Gerado durante execução.	Minimizar dados; não enviar secrets; retenção configurável.

4.2 API ou evento

Item	Definição
Identificador	Base /api/v1. Recursos: auth, agents, objectives, budgets, policies, authorizations, approvals, ledger, outcomes, commerce, audit.
Consumidores	Web app Voytek, agentes internos, SDKs e integrações autorizadas.
Contrato	OpenAPI versionado; DTOs separados de entidades EF.
Erro e idempotência	ProblemDetails; correlationId; Idempotency-Key em operações sensíveis; retry somente seguro.
Compatibilidade	Versionamento por /v1; mudanças incompatíveis exigem nova versão/depreciação comunicada.

4.3 Integrações externas

Sistema	Protocolo	Autenticação	Limite	Plano de falha
LLM Provider	HTTPS REST/SDK	API key/managed secret	Rate/token limits	Timeout, retry seguro, circuit breaker, fallback de provider quando configurado.
Azure	HTTPS	Managed identity/secret	Quota do plano	Alertas, health checks, scale controlado.
E-mail	HTTPS API/SMTP	Secret/OAuth	Quota	Fila/retry; não bloquear transação principal.
Futuros rails financeiros	HTTPS/webhooks	OAuth/mTLS/secret	Por parceiro	Idempotência, reconciliação, webhook assinado, fila e compensação.

5 QUALIDADE, SEGURANÇA E TESTES

5.1 Controles

Área	Controle adotado	Evidência / responsável
Acesso	JWT/Identity, RBAC, tenant isolation, menor privilégio.	Testes de autorização e integração.
Dados	TLS, criptografia gerenciada, secrets fora do código, backup.	Configuração e auditoria.
Entrada	Validação server-side, rate limiting, payload limits, timeout.	Testes e configuração da API.
Financeiro	Policy Engine determinístico, idempotência, reserva de budget, ledger.	Testes unitários/integrados.
IA	LLM somente propõe; nunca autoriza diretamente; contexto mínimo.	Testes de tool flow e revisão.
Disponibilidade	Health checks, logs, métricas, backup, rollback.	Dashboard/runbook.

5.2 Riscos

Risco	Prob./impacto	Mitigação	Gatilho / dono
LLM interpreta intenção incorretamente.	Média / alto	Structured output + Policy Engine determinístico + approval.	Aumento de decisões divergentes / produto.
Cross-tenant data leak.	Baixa / crítico	TenantId, filtros, autorização explícita, testes.	Falha em teste/auditoria / engenharia.
Duplicação de ação por retry.	Média / crítico	Idempotency-Key + transação + outbox.	Duplicidade detectada / engenharia.
Overspending concorrente.	Média / crítico	Reserva transacional e concurrency control.	Saldo negativo/conflito / engenharia.
Custo de IA cresce.	Média / médio	Regras sem LLM quando possível; Luna Low; prompts curtos; caching/roteamento.	Custo por tenant acima da meta / produto.
Dependência externa indisponível.	Média / alto	Timeout, circuit breaker, retry seguro, fila/fallback.	SLO/alerta / operação.

5.3 Plano de testes

Item	Definição
Cobertura	Unidade, integração, arquitetura, contrato, E2E seletivo, carga e segurança conforme maturidade.
Casos críticos	Tenant isolation; budget concorrente; Policy Engine; approvals; idempotência; Shadow Mode; ledger append-only; commerce.
Ambiente	Local/teste com PostgreSQL isolado e dependências externas simuladas.
Liberação	Build limpo, testes críticos verdes, migration validada, rollback definido e secrets verificados.

6 IMPLANTAÇÃO E OPERAÇÃO

6.1 Implantação

Item	Definição
Ambientes	Local -> teste -> homologação/piloto -> produção.
Frontend	React/TypeScript em hosting estático/CDN.
API	Azure Container Apps, Consumption, min replicas=0 e max replicas=1 inicialmente; aumentar conforme tráfego.
Banco	PostgreSQL gerenciado; um cluster no MVP.
Pipeline	GitHub Actions: restore/build/test -> container -> deploy -> validação.
Rollback	Reimplantar imagem anterior; migrations destrutivas exigem estratégia compatível.
Configuração	Environment variables/User Secrets no desenvolvimento; secret store/Key Vault em produção.

6.2 Observabilidade e suporte

Sinal	O que indica	Alerta / primeira ação
HTTP 5xx	Falha de aplicação/infra.	Correlacionar logs; verificar health e dependências.
Latência p95	Degradação.	Ver DB, LLM e endpoints lentos.
DENY/HUMAN_APPROVAL anormal	Policy/configuração ou comportamento de agente.	Revisar policies e eventos.
LLM custo/tokens	Eficiência operacional.	Reduzir contexto, usar regras determinísticas e roteamento.
Falha de DB	Indisponibilidade crítica.	Bloquear efeitos sensíveis e acionar recuperação.

6.3 Manutenção e evolução

Item	Motivo	Prioridade	Próximo passo
Redis	Cache/distributed coordination somente quando necessário.	Baixa inicial	Adicionar após gargalo medido.
Azure Service Bus	Mensageria distribuída.	Média futura	Extrair quando workers/eventos exigirem.
Microserviços	Escala/deploy independente.	Baixa inicial	Extrair módulos por necessidade real.
Key Vault	Gestão madura de secrets.	Alta antes de produção real	Integrar na etapa de hardening.
Pagamentos reais	Evolução comercial/regulatória.	Pós-MVP	Selecionar parceiro regulado e desenhar reconciliação.

7 DECISÕES E REFERÊNCIAS

7.1 Registro de decisões arquiteturais - ADR

ID	Decisão	Contexto / consequência
ADR-001	Modular Monolith	Reduz custo e complexidade; mantém fronteiras para futura extração.
ADR-002	PostgreSQL	Banco relacional robusto para budgets, auditoria e transações.
ADR-003	Shared DB multi-tenant	Operação simples; exige TenantId, autorização e testes de isolamento.
ADR-004	Policy Engine determinístico	LLM não possui autoridade financeira final.
ADR-005	Shadow Mode no mesmo pipeline	Simulação deve reproduzir decisões reais sem efeitos externos.
ADR-006	IEventBus abstrato	In-process inicialmente; broker depois.
ADR-007	Azure Container Apps	Containerização, baixo custo inicial e escala horizontal.
ADR-008	Ledger append-only	Rastreabilidade e correções por compensação.
ADR-009	ILLMProvider	Evita acoplamento a um único fornecedor/modelo.
ADR-010	Idempotência obrigatória	Evita efeitos duplicados em retries e falhas de rede.

7.2 Checklist de aprovação

O MVP só deve avançar para piloto quando: requisitos críticos possuem teste; isolamento multi-tenant foi validado; Shadow Mode não possui caminho para executor real; operações sensíveis são idempotentes; secrets não estão no repositório; logs possuem correlationId; migrations e rollback foram testados; budgets e ledger mantêm consistência sob concorrência.

REFERÊNCIAS

Modelo estrutural e de apresentação baseado no arquivo Model STS fornecido pelo autor. Tecnologias e decisões específicas deste documento refletem o planejamento atual da Voytek.